Lab Documentation: Build a Page of Playing Cards

Platform: freeCodeCamp — Responsive Web Design Certification

Project: Build a Page of Playing Cards

1. Overview & Objectives

The goal of this lab activity was to build a responsive, interactive webpage displaying realistic playing cards using **HTML5** structural markup and **CSS Flexbox** positioning techniques. The implementation emphasizes component-based layout, Flexbox alignment overrides (`align-self`), CSS transforms (180° rotation), dynamic hover states, and color customization for different card suits.

2. Technical Workflow & Implementation

Phase 1: HTML Document Structure (`index.html`)

1. **Document Setup:**
   - Declared `<!DOCTYPE html>` along with language metadata and `UTF-8` encoding.
   - Included a responsive viewport meta tag (`width=device-width, initial-scale=1.0`) and linked the external stylesheet (`styles.css`).

2. **Card Component Architecture:**
   - Wrapped the layout inside a `<main id="playing-cards">` container.
   - Constructed three playing card instances (`div.card`), each divided into three structural sub-sections: `.left`, `.middle`, and `.right`:

     - **Card 1 (Ace of Spades):** Configured with top-left rank/suit (`A` & `♠`), central suit symbol (`♠`), and bottom-right rank/suit (`♠` & `A`).
     - **Card 2 (2 of Clubs):** Configured with top-left (`2` & `♣`), two centered suit symbols (`♣`), and bottom-right (`♣` & `2`).
     - **Card 3 (3 of Hearts):** Configured with top-left (`3` & `♥`), three centered suit symbols (`♥`), and bottom-right (`♥` & `3`).

Phase 2: CSS Styling & Alignment (`styles.css`)

1. **Global Reset & Canvas Configuration:**
   - Styled `body` with a light background (`#eef2f5`), full viewport height (`min-height: 100vh`), standard system font stack, and `box-sizing: border-box`.

   - Applied `display: flex`, `justify-content: center`, `flex-wrap: wrap`, and a `gap: 20px` on `#playing-cards` to cleanly align and space the cards.

2. **Card Container Component (`.card`):**
    - Configured fixed card dimensions (`width: 200px`, `height: 300px`), rounded corners (`border-radius: 12px`), a crisp white background (`#ffffff`), and subtle box shadows (`0 4px 12px rgba(0, 0, 0, 0.08)`).
    - Used `display: flex` and `justify-content: space-between` to distribute the `.left`, `.middle`, and `.right` sections across the width of the card.
    - Set `user-select: none` to prevent text-selection highlights on UI interaction.
3. **Sub-Section Layout & Alignment Overrides:**
    - **Left Corner (`.left`):** Set `align-self: flex-start` with a vertical flex layout (`flex-direction: column`) to anchor the rank and suit to the top-left.
    - **Center Section (`.middle`):** Set `align-self: center` with `justify-content: center` and a vertical gap (`gap: 8px`) to center the card pip symbols.
    - **Right Corner (`.right`):** Set `align-self: flex-end` with a vertical flex layout.
    - **180° Inversion:** Applied `transform: rotate(180deg)` to `.right` to flip the bottom-right rank and suit upside down, mirroring traditional playing card design.
4. **Hover Micro-Interactions & Suit Color Modifiers:**
    - **Interaction:** Defined `.card:hover` with `transform: translateY(-4px)` and an enhanced `box-shadow` for smooth elevation feedback on focus/hover.
    - **Color Customization:** Used class grouping (`.card.red .left, .card.red .middle, .card.red .right`) to set suit colors to red (`#dc2626`) for heart cards.

3. Knowledge Gained & Key Takeaways

- **Flexbox Item Alignment (`align-self`):** Mastered using `align-self` (`flex-start`, `center`, `flex-end`) to independently position child elements along the cross-axis within a single parent container.
- **CSS Spatial Transformations:** Gained practical experience applying `transform: rotate(180deg)` to invert UI components cleanly without altering DOM ordering.
- **Component-Driven Styling:** Structured CSS using shared base classes (`.card`, `.left`, `.middle`, `.right`) alongside modifier classes (`.card.red`) for maintainable, reusable code.
- **UI Polish & Micro-Interactions:** Leveraged subtle drop shadows, smooth elevation transitions (`translateY`), and disabled text selection (`user-select: none`) to achieve a realistic component feel.

CODE:











PREVIEW:

